

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1401

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration :60 Mins
On Class
Total Marks:50

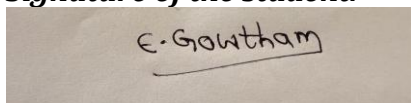
Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I Enamala Gowtham/192311190(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a piece of paper with the handwritten signature "E. Gowtham" in black ink. The signature is written in a cursive style and is underlined.

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Q1. Smart Retail Billing System

Concepts Covered: Intermediate Languages, Declarations, Assignment Statements

Problem Statement

A retail store billing system calculates the final payable amount based on product price, quantity purchased, and discount percentage.

```
total_cost = price * quantity
discount_value = total_cost * discount
final_bill = total_cost - discount_value
```

Tasks:

- (i) Write appropriate declarations for all variables with suitable data types.
- (ii) Generate the three-address intermediate code for the given statements.
- (iii) Represent the same using quadruples, triples, and indirect triples.
- (iv) Identify possible optimization opportunities in the generated intermediate code.
- (v) Generate the optimized intermediate code and justify the improvements. (10 Marks)

Q2. University Scholarship Eligibility System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

A student is eligible for a scholarship if:

CGPA > 8.5
AND (Family Income < 300000 OR Sports Quota = TRUE)

Tasks:

- (i) Formulate the Boolean expression representing the eligibility criteria.
- (ii) Generate intermediate code using short-circuit evaluation.

(iii) Apply backpatching to resolve forward jumps.

(iv) Construct the final labelled intermediate code.

(v) Draw the corresponding control-flow graph. (10 Marks)

Q3. Smart Parking Management System

Concepts Covered: Case Statements, Intermediate Languages

Problem Statement

A parking management system assigns parking zones based on vehicle type.

```
switch(vehicle_type)
{
    case 1: TWO_WHEELER;
    case 2: CAR;
    case 3: BUS;
    case 4: EMERGENCY_VEHICLE;
}
```

Tasks:

(i) Convert the switch statement into three-address intermediate code using conditional jumps.

(ii) Generate an equivalent jump-table implementation.

(iii) Represent the intermediate code using quadruples.

(iv) Compare conditional branching and jump-table implementations.

(v) Recommend the best approach for a large-scale parking system with justification.

(10 Marks)

Q4. Online Food Delivery Processing System

Concepts Covered: Procedure Calls, Activation Records, Intermediate Code

Problem Statement

An online food delivery platform executes the following function calls:

```
amount = calculateBill(order)
applyCoupon(amount)
generateInvoice(amount)
```

Tasks:

- (i) Generate intermediate code for the procedure calls.
- (ii) Illustrate parameter passing and return value handling.
- (iii) Design the activation record (stack frame) for each procedure call.
- (iv) Show the sequence of call and return operations using stack diagrams.
- (v) Explain how local variables, parameters, and return addresses are stored in the activation record. (10 Marks)

Q5. Smart Industrial Safety Monitoring System

Concepts Covered: Assignment Statements, Boolean Expressions, Backpatching, Procedure Calls

Problem Statement

An industrial safety system activates an emergency alarm if:

Temperature > 100
AND (Pressure > 80 OR GasLeak = TRUE)

When the condition is satisfied, the system invokes:

```
activateAlarm()  
sendNotification()
```

Tasks:

- (i) Write the Boolean expression representing the safety condition.
- (ii) Generate intermediate code using short-circuit evaluation.
- (iii) Apply backpatching to resolve all jump targets.
- (iv) Generate intermediate code for the procedure calls executed when the condition is true.
- (v) Represent the complete control flow using labelled instructions and explain the execution sequence. (10 Marks)

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q1. Smart Retail Billing System

(i) Declarations

Suitable declarations are:

```
int quantity;  
float price;  
float discount;  
float total_cost;  
float discount_value;  
float final_bill;
```

Here:

- price $\rightarrow$ product price
- quantity $\rightarrow$ number of products
- discount $\rightarrow$ discount percentage/fraction
- total_cost $\rightarrow$ price $\times$ quantity
- discount_value $\rightarrow$ calculated discount
- final_bill $\rightarrow$ final payable amount

(ii) Three-Address Intermediate Code

Given:

```
total_cost = price * quantity  
discount_value = total_cost * discount  
final_bill = total_cost - discount_value
```

Three-address code:

```
t1 = price * quantity  
t2 = t1 * discount  
t3 = t1 - t2  
final_bill = t3
```

(iii) Quadruples, Triples and Indirect Triples

Quadruples:

No.	Operator	Arg1	Arg2	Result
1	*	price	quantity	t1
2	*	t1	discount	t2
3	-	t1	t2	t3
4	=	t3	—	final_bill

Triples:

No.	Operator	Arg1	Arg2
0	*	price	quantity
1	*	(0)	discount
2	-	(0)	(1)
3	=	(2)	final_bill

Indirect Triples:

The indirect triple table contains pointers to the triple entries:
Pointer Table

0 $\rightarrow$ Triple 0

- 1 → Triple 1
- 2 → Triple 2
- 3 → Triple 3

This allows the order of execution to be changed by modifying pointers instead of moving the triples themselves.

(iv) Optimization Opportunities

Possible optimization techniques are:

1. Eliminate unnecessary temporary variables.
2. Combine arithmetic expressions.
3. Reduce the number of intermediate values.
4. Use algebraic simplification.

The original expression is:

`final_bill = total_cost - (total_cost * discount)`

It can be written as:

`final_bill = total_cost * (1 - discount)`

(v) Optimized Intermediate Code

`t1 = price * quantity`

`t2 = 1 - discount`

`t3 = t1 * t2`

`final_bill = t3`

This form avoids the separate `discount_value` variable and represents the calculation more directly. It reduces the number of temporary values and makes the code easier for a compiler to optimize.

Q2. University Scholarship Eligibility System

The question specifies that eligibility requires CGPA greater than 8.5 and either low family income or sports quota.

(i) Boolean Expression

Eligibility =

`(CGPA > 8.5) AND`

`((FamilyIncome < 300000) OR (SportsQuota == TRUE))`

(ii) Intermediate Code Using Short-Circuit Evaluation

1. `if CGPA > 8.5 goto 3`

2. `goto 8`

3. `if FamilyIncome < 300000 goto 7`

4. `if SportsQuota == TRUE goto 7`

5. `goto 8`

7. `ScholarshipEligible = TRUE`

8. `ScholarshipEligible = FALSE`

(iii) Backpatching

Initially, the forward jumps do not have their final destinations.

After backpatching:

1. `if CGPA > 8.5 goto 3`

2. `goto 8`

3. `if FamilyIncome < 300000 goto 7`

4. `if SportsQuota == TRUE goto 7`

5. `goto 8`

7. `ScholarshipEligible = TRUE`

8. `ScholarshipEligible = FALSE`

The true branches of the OR condition are patched to statement 7.

The false branches are patched to statement 8.

(iv) Final Labelled Intermediate Code

L1:

```
if CGPA > 8.5 goto L3
goto L8
```

L3:

```
if FamilyIncome < 300000 goto L7
if SportsQuota == TRUE goto L7
goto L8
```

L7:

```
ScholarshipEligible = TRUE
goto L9
```

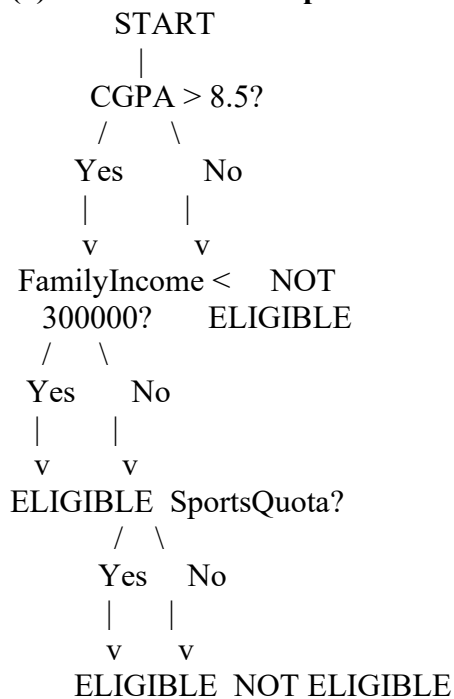
L8:

```
ScholarshipEligible = FALSE
```

L9:

```
END
```

(v) Control-Flow Graph



Short-circuit evaluation avoids checking the remaining conditions when the result is already known.

Q3. Smart Parking Management System

The question requires conversion of the switch statement into conditional jumps, a jump table, quadruples, and comparison of both approaches.

(i) Three-Address Code Using Conditional Jumps

1. if vehicle_type == 1 goto L1
2. if vehicle_type == 2 goto L2
3. if vehicle_type == 3 goto L3
4. if vehicle_type == 4 goto L4
5. goto L5

L1:
TWO_WHEELER
goto L6

L2:
CAR
goto L6

L3:
BUS
goto L6

L4:
EMERGENCY_VEHICLE
goto L6

L5:
INVALID_VEHICLE_TYPE

L6:
END

(ii) Equivalent Jump-Table Implementation

1. if vehicle_type < 1 goto Ldefault
2. if vehicle_type > 4 goto Ldefault
3. goto JumpTable[vehicle_type]

JumpTable:
1 → L1
2 → L2
3 → L3
4 → L4

L1:
TWO_WHEELER
goto Lend

L2:
CAR
goto Lend

L3:
BUS

goto Lend

L4:

EMERGENCY_VEHICLE

goto Lend

Ldefault:

INVALID_VEHICLE_TYPE

Lend:

END

(iii) Quadruples

No.	Operator	Arg1	Arg2	Result
-----	----------	------	------	--------

1	==	vehicle_type 1		L1
---	----	----------------	--	----

2	==	vehicle_type 2		L2
---	----	----------------	--	----

3	==	vehicle_type 3		L3
---	----	----------------	--	----

4	==	vehicle_type 4		L4
---	----	----------------	--	----

5	goto	—	—	L5
---	------	---	---	----

The labels L1–L4 contain the corresponding vehicle-type actions.

(iv) Comparison

Conditional Branching

Jump Table

Checks conditions one by one	Directly selects destination
------------------------------	------------------------------

Simple to implement	Faster for many cases
---------------------	-----------------------

Suitable for few cases	Suitable for many dense cases
------------------------	-------------------------------

Requires multiple comparisons	Requires table storage
-------------------------------	------------------------

Good for sparse values	Best for consecutive values
------------------------	-----------------------------

(v) Recommendation

For a **large-scale parking management system with many consecutive vehicle types**, a **jump table** is recommended.

The reason is that the jump table can directly identify the required code block instead of comparing the vehicle type against every case one by one. This can improve execution speed when there are many case values.

Q4. Online Food Delivery Processing System

The question involves procedure calls, parameter passing, return values, activation records, and stack operations.

Given:

```
amount = calculateBill(order)
applyCoupon(amount)
generateInvoice(amount)
```

(i) Intermediate Code

```
param order
amount = call calculateBill, 1
```

```
param amount
call applyCoupon, 1
```

```
param amount
call generateInvoice, 1
```

If `applyCoupon()` returns a modified amount, it can instead be represented as:

```
param amount
amount = call applyCoupon, 1
```

(ii) Parameter Passing and Return Value

For `calculateBill(order)`:

```
param order
amount = call calculateBill, 1
```

Here:

- `order` is passed as a parameter.
- `calculateBill` executes.
- The returned value is stored in `amount`.

For `applyCoupon(amount)`:

```
param amount
call applyCoupon, 1
```

The current amount is passed to the procedure.

For `generateInvoice(amount)`:

```
param amount
call generateInvoice, 1
```

The amount is passed to generate the invoice.

(iii) Activation Record

A typical activation record contains:

```
+-----+
| Parameters      |
+-----+
| Return Address  |
+-----+
| Control Link    |
+-----+
| Access Link     |
+-----+
| Saved Registers |
+-----+
| Local Variables |
+-----+
| Temporary Values|
+-----+
```

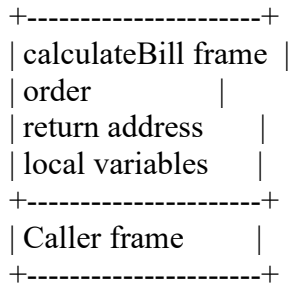
Each procedure gets its own activation record when it is called.

(iv) Stack Diagram

Initially:

Caller

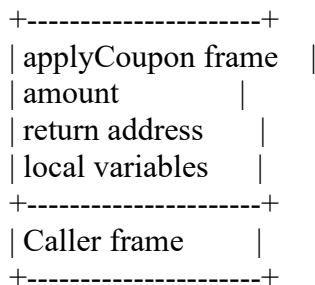
When calculateBill() is called:



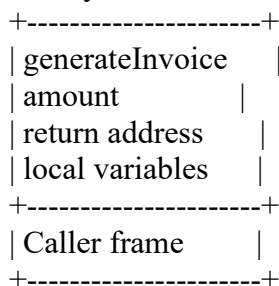
After it returns:

amount = returned value

Then applyCoupon():



Finally:



After the procedure completes, its activation record is removed from the stack.

(v) Storage in Activation Record

- **Parameters:** Store values passed to the procedure.
- **Return address:** Stores where execution should continue after the procedure returns.
- **Local variables:** Store variables declared inside the procedure.
- **Temporaries:** Store intermediate calculations.
- **Saved registers:** Preserve required processor register values.
- **Control link:** Connects the current activation record to the caller's activation record.

Q5. Smart Industrial Safety Monitoring System

The question specifies that the alarm condition is $\text{Temperature} > 100 \text{ AND } (\text{Pressure} > 80 \text{ OR GasLeak} = \text{TRUE})$, followed by two procedure calls.

(i) Boolean Expression

SafetyCondition =

$(\text{Temperature} > 100) \text{ AND}$

$((\text{Pressure} > 80) \text{ OR } (\text{GasLeak} == \text{TRUE}))$

(ii) Intermediate Code Using Short-Circuit Evaluation

1. if $\text{Temperature} > 100$ goto L3

2. goto Lfalse

L3:

3. if $\text{Pressure} > 80$ goto Ltrue

4. if $\text{GasLeak} == \text{TRUE}$ goto Ltrue

5. goto Lfalse

Ltrue:

6. call activateAlarm

7. call sendNotification

8. goto Lend

Lfalse:

9. goto Lend

Lend:

10. END

The system first checks temperature.

If temperature is not greater than 100, the remaining conditions do not need to be checked.

If temperature is greater than 100, either high pressure or gas leakage is sufficient to activate the alarm.

(iii) Backpatching

The forward jumps are resolved as follows:

$\text{Temperature} > 100$

|
| True
v
L3

$\text{Pressure} > 80$

|
| True
v
Ltrue

$\text{GasLeak} == \text{TRUE}$

|
| True
v
Ltrue

False branches go to Lfalse.

Therefore:

1. if $\text{Temperature} > 100$ goto L3

2. goto Lfalse

3. if Pressure > 80 goto Ltrue
4. if GasLeak == TRUE goto Ltrue
5. goto Lfalse

(iv) Intermediate Code for Procedure Calls

When the safety condition is true:

Ltrue:

- call activateAlarm
- call sendNotification
- goto Lend

The first procedure activates the emergency alarm.

The second procedure sends a notification.

(v) Complete Labelled Control Flow

L1:

- if Temperature > 100 goto L3
- goto Lfalse

L3:

- if Pressure > 80 goto Ltrue
- if GasLeak == TRUE goto Ltrue
- goto Lfalse

Ltrue:

- call activateAlarm
- call sendNotification
- goto Lend

Lfalse:

- goto Lend

Lend:

END

Execution Sequence

1. Check whether Temperature > 100.
2. If false → no emergency action.
3. If true → check Pressure > 80.
4. If pressure is greater than 80 → activate the alarm.
5. If pressure is not greater than 80 → check GasLeak.
6. If GasLeak == TRUE → activate the alarm.
7. When the condition is true, call:

activateAlarm()

sendNotification()

8. Execution then reaches Lend.